

Documento Final de Entrega

Tech Challenge - Fase 2 - Grupo NumberOne

Este documento consolida os itens obrigatórios da entrega final da Fase 2 em formato estruturado, facilitando a validação dos requisitos, dos artefatos de infraestrutura, dos fluxos de CI/CD, dos links de apoio e dos pontos pendentes para fechamento antes da submissão no portal do aluno.

Informações da Entrega

Item	Detalhamento
Nome do grupo	NumberOne
Participantes e usernames no Discord	Anderson - anderson_muniz Julio - julioosilva97 Matheus - .msoaresli Marcelo - marcelodot.i
Link da documentação	Documentacao tecnica no repositorio: README.md, doc/, infra/ e .k8s/.
Link do repositório	https://github.com/MarceloGilos/PosTech15SOAT.git
Link do vídeo demonstrativo	https://drive.google.com/file/d/1n5WM2rpurVWOdH4W26HUwON4WfsVHgNI/view?usp=sharing
Collection das APIs	Swagger/OpenAPI - http://localhost:8080/swagger-ui/index.html

Referências de Apoio

- Repositório do projeto com código-fonte, Dockerfile, docker-compose, workflows, manifests Kubernetes e scripts Terraform.
- Documentação técnica em README.md, doc/README.md, doc/testes/README.md e doc/security/README.md.
- Infraestrutura como código em infra/ e documentação dos módulos Terraform em infra/modules/*/README.md.
- Manifests Kubernetes em .k8s/ para aplicação, banco de dados, Mailpit, ConfigMaps, Secrets, Services, Deployments, PVC e HPA.
- Diagramas de arquitetura, componentes, CI/CD e infraestrutura em doc/diagrams.

Disposição do Repositório - Fase 2

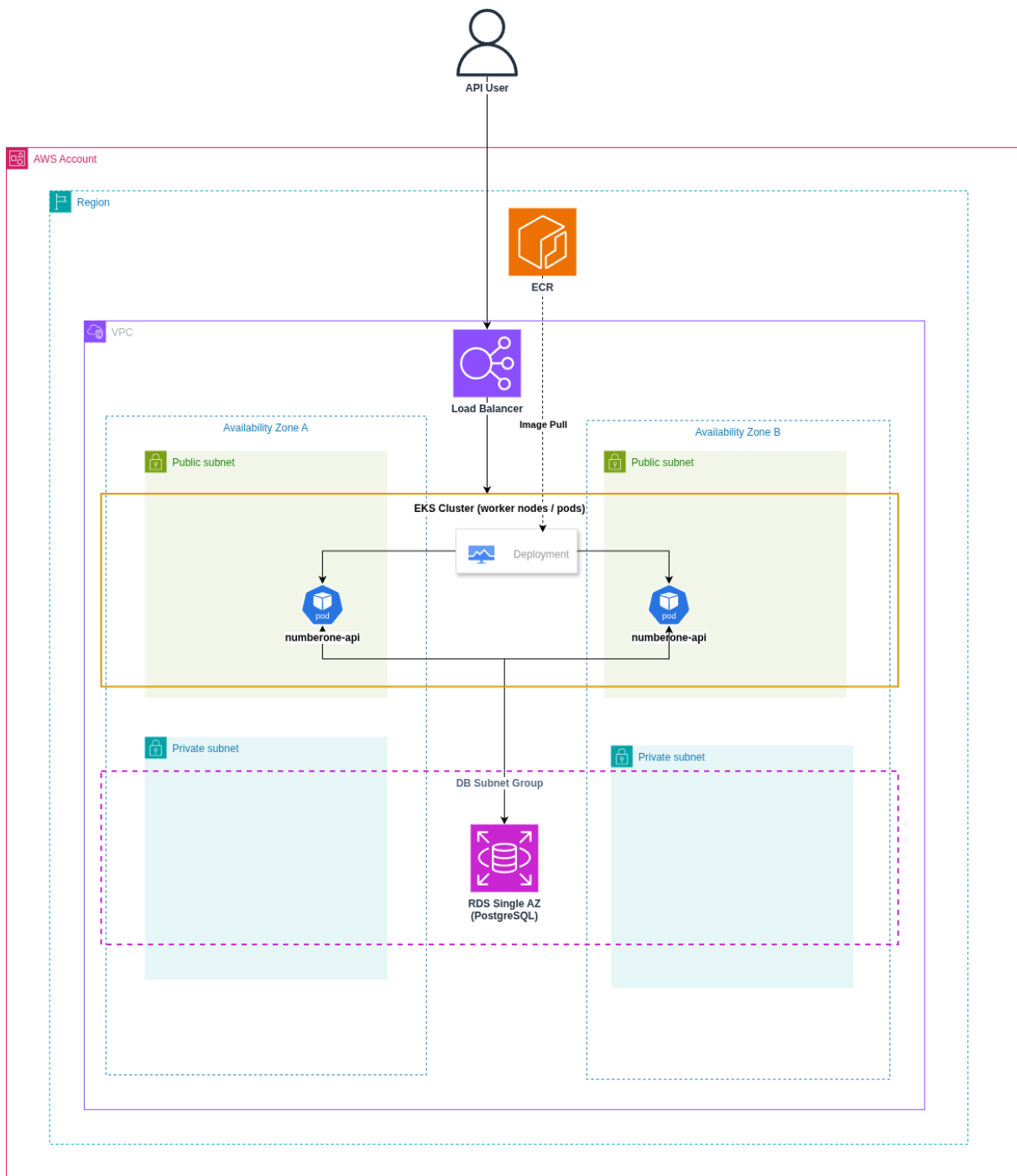
A estrutura do repositório foi organizada para separar código de aplicação, documentação, infraestrutura como código, manifests Kubernetes, automação de CI/CD e evidências de qualidade. Essa organização facilita a rastreabilidade dos requisitos da Fase 2 e a validação dos artefatos entregues.

Estrutura	Finalidade
src/main/java/br/com/fiap/numberone	Código-fonte da API organizado por módulos e camadas de arquitetura.
src/test	Testes unitários, integração e Cucumber/E2E.
Dockerfile	Imagem containerizada da aplicação Java/Spring Boot.
docker-compose.yml	Ambiente local com aplicação, PostgreSQL e

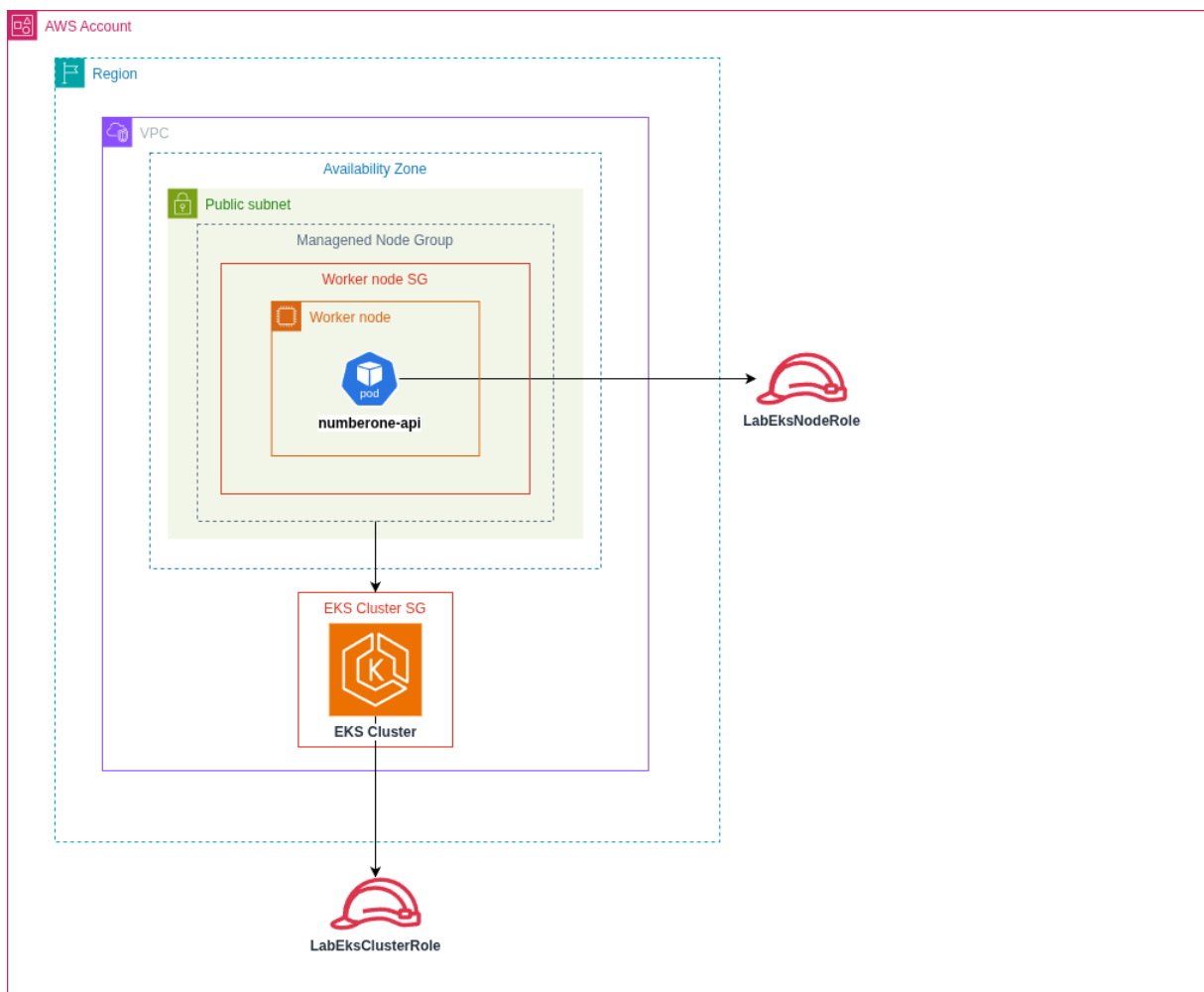
	Mailpit.
.k8s/	Manifests Kubernetes para app, banco, Mailpit, ConfigMaps, Secrets, Services, Deployments, PVC e HPA.
.github/workflows/	Pipelines de CI, deploy local do banco e deploy local da API usando runner self-hosted.
infra/	Terraform para VPC, EKS, RDS e ECR, além de scripts AWS de deploy/destruição.
doc/diagrams/	Diagramas de arquitetura geral, EKS, VPC, componentes C4/Mermaid e fluxo de GitHub Actions.
doc/	Documentação técnica, testes, segurança, módulos, linguagem ubíqua e documentação de equipe.

Desenho da Arquitetura Proposta

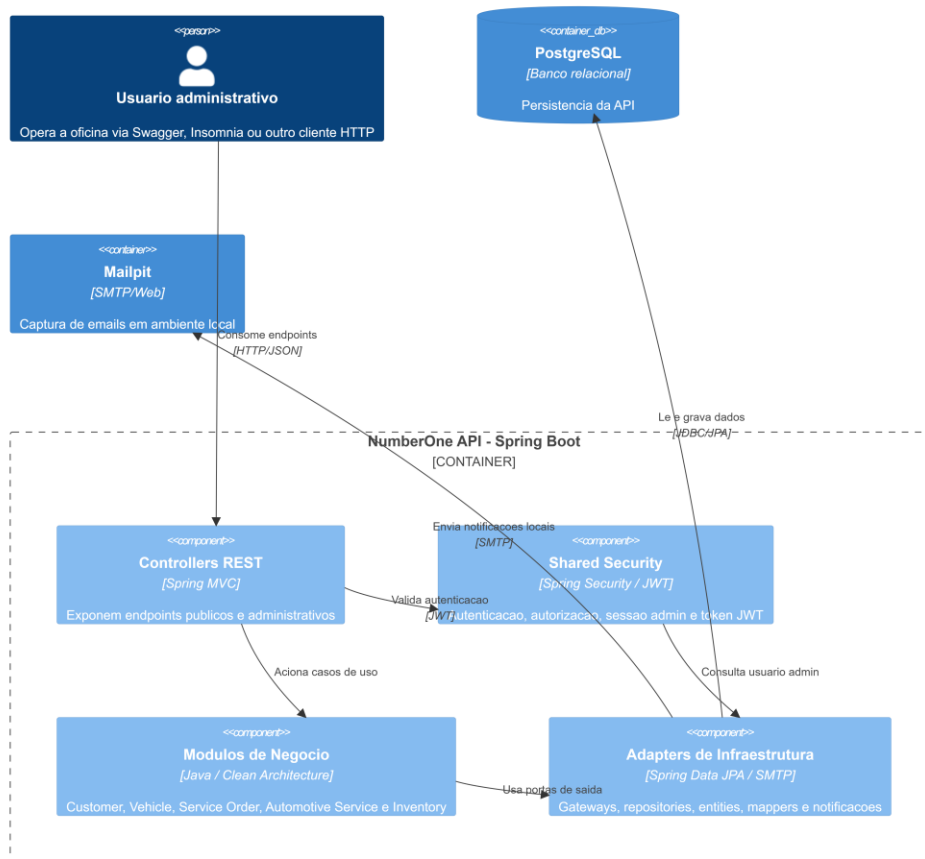
Os desenhos abaixo representam os recursos escolhidos para a solução: aplicação Spring Boot, Kubernetes/EKS, banco PostgreSQL/RDS, registry ECR, rede AWS e componentes internos da API.



Arquitetura geral da solucao - doc/diagrams/terraform/numberone-geral.drawio.png



Arquitetura do cluster Kubernetes/EKS - doc/diagrams/terraform/numberone-eks.drawio.png



Visao C4 dos componentes - doc/diagrams/componente/Visao C4 Component - Geral.png

Leitura Recomendada da Documentação

- Primeiro, ler o README.md da raiz para visão geral, execução local, autenticação, endpoints e testes.
- Em seguida, consultar .k8s/README.md para execução em Kubernetes local e estrutura dos manifests.
- Depois, consultar infra/README.md e os READMEs dos módulos Terraform para entender VPC, EKS, RDS e ECR.
- Para arquitetura de componentes, consultar doc/diagrams/componente/componentes.md e as imagens geradas.
- Por fim, validar testes e segurança em doc/testes/README.md e doc/security/relatorio-vulnerabilidades.md.

Complemento de CI/CD e Infraestrutura

A pipeline local foi separada em três workflows: CI, CD Local - Banco de Dados e CD Local - API. O CI executa validação Maven, build, testes unitários e validação dos manifests. O CD do banco aplica namespace, PostgreSQL e Mailpit. O CD da API realiza build da imagem Docker, carrega a imagem no Minikube quando disponível, aplica ConfigMap, Secret, Service, Deployment e HPA, e aguarda o rollout da aplicação.

O Terraform não precisa necessariamente executar dentro da pipeline da aplicação para atender ao enunciado. O requisito de IaC está contemplado pelos scripts em infra/, enquanto o CI/CD atua sobre build, testes, imagem Docker e deploy no cluster Kubernetes. Opcionalmente, uma pipeline separada de infraestrutura pode ser criada para automatizar terraform plan/apply.

Complemento sobre Kustomize e runner self-hosted

A estrutura Kubernetes foi organizada com Kustomize: .k8s/base concentra os manifests comuns e .k8s/overlays separa as variações local e AWS. Os workflows locais aplicam os overlays local/database e local/api, reduzindo duplicação de YAML e deixando explícito o que muda por ambiente.

A execução local do deploy via GitHub Actions depende de runner self-hosted no Windows com label runner-windows-0002. Esse runner fica conectado ao GitHub aguardando jobs; quando uma action compatível é disparada, ele baixa o código na pasta _work e executa as steps contra o Docker Desktop e o Minikube da máquina do desenvolvedor.